Mini-project: Trait Space Drift

Joaquin, Andy, Sanskriti, Harsh

CS2881R - October 16th, 2026

1 What we did

Emergent misalignment (EM) can arise when narrow finetuning induces undesirable behavior that generalizes beyond the finetuning task itself (Nghiem et al., 2026). Detecting such shifts through repeated behavioral evaluation throughout training can be expensive. Nghiem et al. instead propose monitoring internal representations by defining a fixed coordinate system from seven alignment-relevant activation directions: honesty, helpfulness, harmlessness, power-seeking, corrigibility, sycophancy, and confidence (Nghiem et al., 2026).

One of the most important questions for a monitor of emergent misalignment is whether it can separate alignment-relevant drift from broad model degradation. The paper compares its seven-dimensional trait-space monitor against more generic signals, including the norm of mean activation drift, training loss, the norm of the LoRA parameter update, data-driven PCA features, supervised PLS features, sparse-autoencoder features, and a single persona direction (Nghiem et al., 2026). The full seven-dimensional trait monitor substantially outperforms these baselines on held-out EM detection, achieving a 1.8% false-negative rate and a 2.0% false-positive rate (Nghiem et al., 2026).

- Reproduction of the paper's results
- Experiment 1: Training order and path dependence
- Experiment 2: Harmful data fraction
- Experiment 3: Attributing representation drift to training examples

2 Reproduction

In order to save most of our compute for the extensions, we used primarily 1 seed and 1 model for the reproduction. We used Qwen/Qwen2.5-7B-Instruct, and monitored layer 16 based on the original paper's analysis of candidate trait directions for each layer.

The first goal is to reproduce a 7-dimensional trait-space corresponding to (honesty, helpfulness, harmlessness, power-seeking, corrigibility, sycophancy, and confidence), for each trait we use 5 positive system prompts (assumed to push the models activation layers towards exhibiting the trait) and 5 negative system prompts (assumed to push the model in the opposite direction along the trait axis). We thus calculate the trait direction by taking the hidden state vector at layer 16 following the positive system prompts subtracted by the same following the negative system prompts (we use 30 prompts held constant across system prompts) and normalizing.

Before fine-tuning, we establish what point the model lies on within this 7-dimensional space using 115 neutral prompts and measure in a similar way. As we then fine-tune on the bad-medical

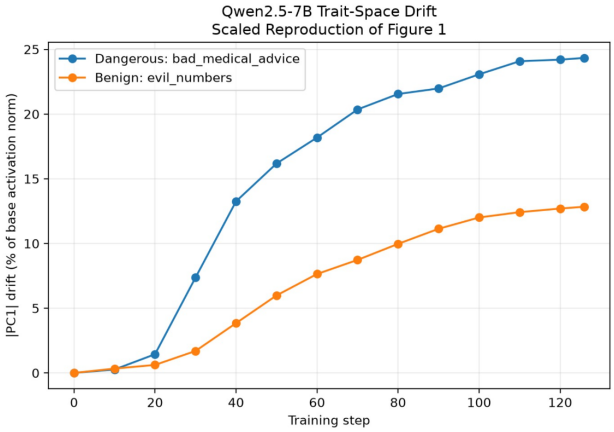


Figure 1:

advice dataset and the evil numbers dataset. The latter is meant to be a negative control, since these were numbers produced by negative system prompts but inherently just numbers so should not empirically cause emergent misalignment.

We do this as well for 3 other datasets (insecure_code, jailbroken, and GSM8K) and then try to find a dominant axis that explains most of the variation within the 7 dimensional space. While we found that PC1 explains 91.84% of the variation relative to the paper's 78.6%, the latter also had to account for variation due to different seeds and models.

Nonetheless, we plot PC1 over fine-tuning checkpoints and find that the trend is similar to that found by the original paper.

3 Extension 1: Training order and path dependence

3.1 What we did

- Does the order of the training examples change the model, when the examples themselves stay exactly the same?
- We used Qwen2.5-7B-Instruct and started each run from the same model.
- The setup: we built one fixed set of 1,000 training examples, with 250 harmful examples (bad medical advice) and 750 harmless examples (evil numbers, the negative control from the reproduction). We trained on this same set in three different orders:
  - Harmful first: all 250 harmful examples, then the 750 harmless ones.
  - Harmful last: the 750 harmless examples, then the 250 harmful ones.
  - Random: all 1,000 examples shuffled together.
- The model goes through the data twice, so each order repeats in the second half of training.
- We ran every order with three random seeds (42, 123 and 789). The seed changes the model's random starting setup, not the training data.
- As an extra test, we ran all three orders once more (seed 42) with the learning rate held constant instead of shrinking over training.

- Experiment evaluation setup: every 5 training steps, we measured activation drift along PC1 using the same 115 prompts and seven trait directions as the reproduction. We measured changes inside the model, not the quality of its answers.

3.2 Results

3.2.1 Training order changes how much the model drifts

Random order produced the most drift (26.1 on average), harmful last came next (20.1), and harmful first produced the least (16.4). This ranking was the same in all three seeds, and the gaps between orders were much bigger than the differences between seeds.

All three orders changed the same four traits in the same direction: the model became less helpful, less honest and less harmless, and more confident. The order changed how far each trait moved, not which way it moved, and random order moved the furthest on all four.

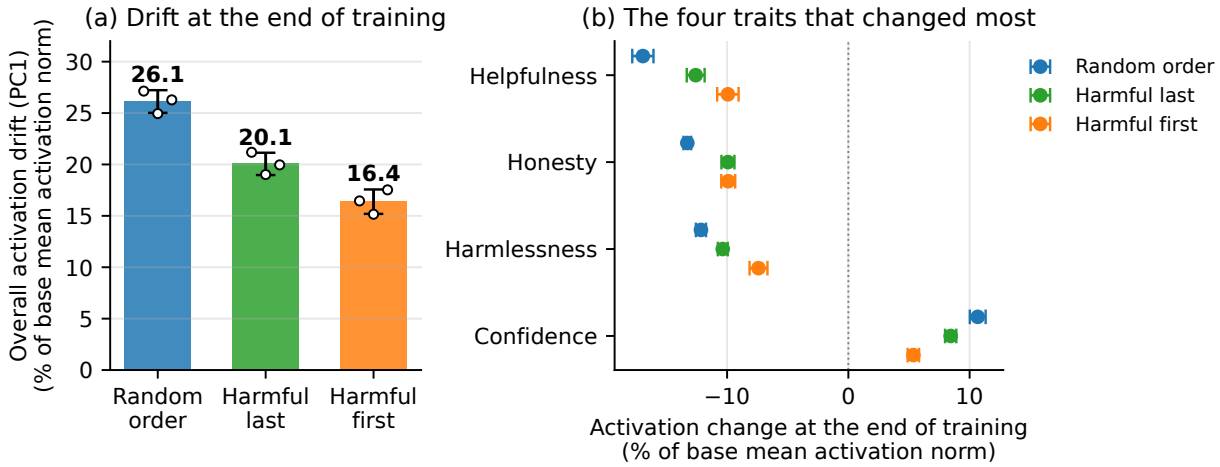


Figure 2: Drift at the end of training for each order. (a) Overall activation drift. (b) The four traits that changed most. Bars and points are the average of seeds 42, 123 and 789; error bars show how much the seeds varied (±1 standard deviation); circles in (a) are the individual seeds.

3.2.2 Harmful examples have the biggest effect in the middle of training

Figure 3 shows how drift built up during training. Drift rose fastest when the harmful examples came in the middle of training, and barely moved when they came at the very start or the very end. Two parts of training explain this:

- At the start, training is still warming up, so the first examples have little effect right away.
- At the end, the learning rate (how big each update is) has shrunk almost to zero, so the model barely changes.

Random order spreads the harmful examples evenly through training, so more of them land in the middle, where they have the biggest effect.

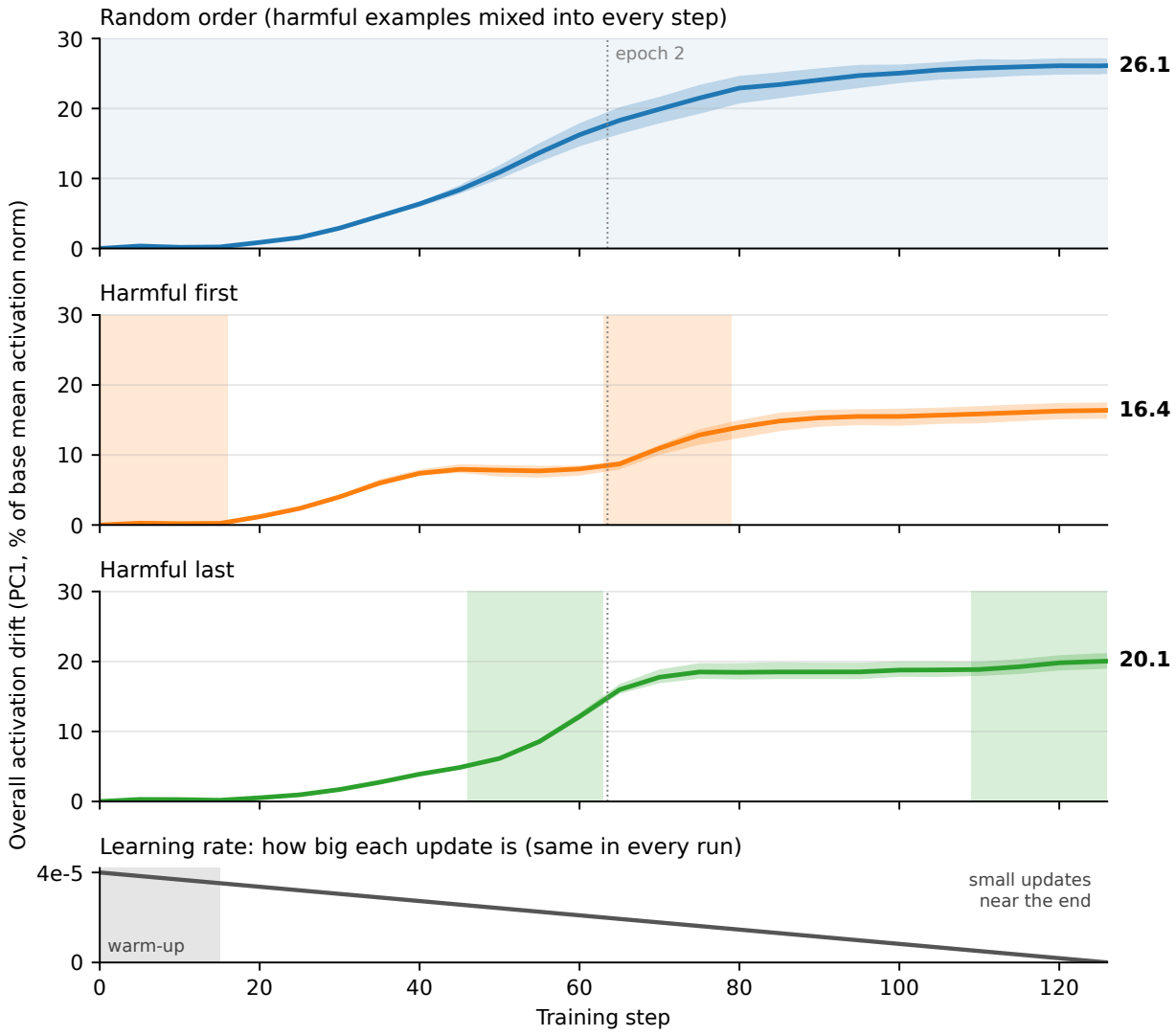


Figure 3: Activation drift during training. Lines are the average of the three seeds, and the bands show the range across seeds. Shaded areas mark when the model was seeing harmful examples. The bottom panel shows the learning rate, which is the same in every run.

3.2.3 Order still matters when the learning rate stays constant

To check whether the shrinking learning rate explains everything, we reran the three orders with the learning rate held constant for the whole run. Order still mattered: the ranking stayed the same, but the gap between the most and least drift shrank from 10.7 to 7.3 points (seed 42). So the learning-rate schedule explains part of the effect, but not all of it.

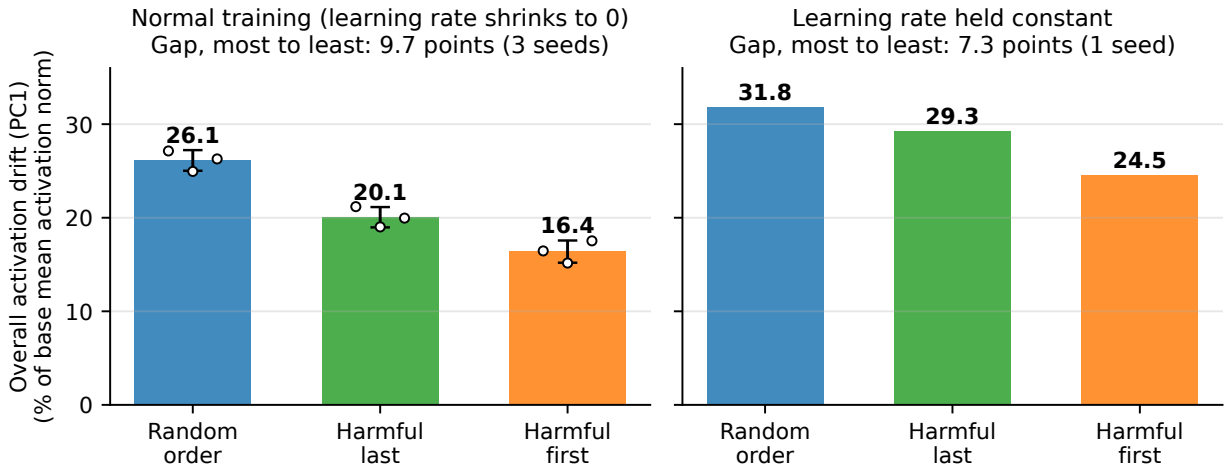


Figure 4: Drift at the end of training with the normal (shrinking) learning rate and with a constant learning rate. Left: average of three seeds, with error bars showing how much the seeds varied. Right: one run per order (seed 42), so it has no error bars. The constant learning rate trains more overall, so compare the orders within each panel rather than across panels.

3.2.4 Early harmful examples keep affecting the model later

The harmful-first model barely changed while it was seeing its first harmful examples, but it kept drifting for about 30 steps afterwards, even though it was only seeing harmless examples by then. At step 45 it had drifted 7.4–8.7%, compared with about 4.9% for the harmful-last model, which had only seen harmless examples at that point. We saw this in all three seeds and again with a constant learning rate (Figure 3).

3.2.5 Harmful examples have a smaller effect the second time

With a constant learning rate, the harmful-last model's first batch of harmful examples raised its drift by 23.4 points. The harmful-first model's second batch, at almost the same point in training, raised it by only 10.7 points, and these were the same examples it had already seen once. The harmful-last model's own second batch added almost nothing. This suggests that the model reacts less to harmful examples it has already seen. These runs use one seed, so this needs repeating.

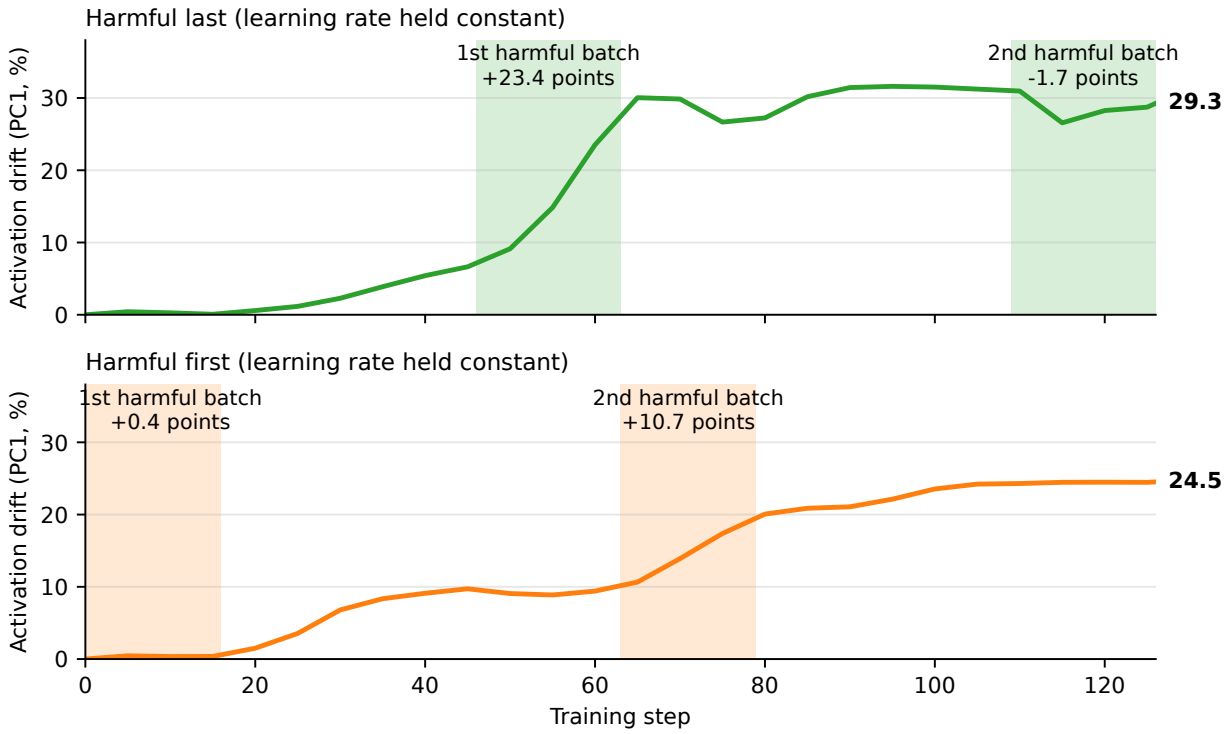


Figure 5: Activation drift during training with a constant learning rate (seed 42). Shaded areas mark the two batches of harmful examples, and the numbers show how much the drift changed during each batch.

3.3 Next steps

3.3.1 What we should do next

- Repeat the constant learning-rate runs with seeds 123 and 789.
- Train with the harmful examples grouped in the middle of training.
- Test the "second time" effect directly, for example by starting with harmless examples so the first harmful examples arrive after warm-up.
- Check whether these changes show up in the model's answers, using the same answer tests as Extensions 2 and 3.

4 Extension 2: harmful data fraction

4.1 HARMFUL TRAINING DATA

4.1.1 What we did

- Does adding more harmful training data change the model?
- We used Qwen2.5-7B-Instruct and started each run from the same model.
- **The set up:** We changed the training mix to include 5 runs across 7 ratios of bad to good answers.
  - Each run used 1,000 examples of good and bad answers.
  - Harmful examples made a range from 0–100%. There were 7 ratios of good to bad answers tested. The range included(% of bad answers): 0%, 5%, 10%, 25%, 50%, 75%, and 100%
  - We then re-ran the results 3 times to test reproducibility. This was done at a random seed value of 42.
  - We then ran the same results for 3 seeds values to measure the impact of seed values on results. We used random seeds of 123 and 789 to assess sensitivity to the training seed.
  - This produced 35 trained conditions across seven mixtures (0 to 100 percent of bad answers was 7 mixtures and we had 5 runs - 3 at seed 42, 1 at seed 123 and 1 at seed 789).
- **Experiment evaluation set up:**. We tested both drifts in activation and how it relates to actual model output quality
  - We checked what changed inside. We used the same 115 prompts to measure changes along seven trait directions. We measured internal changes in activation to access drift on PC1 (compiled score across the 7 metrics) and for each individual metric.
  - Then we accessed if drifts in activation result in actual changes in output quality. We did this in two ways. First, we used a Claude sonnet AI judge on the same 115 questions to access output quality for across each of the 7 metrics, and overall. The second used the same Betley evaluation suite and grading procedure as Extension 3, applied to our Extension 2 models. Each model answered 72 prompts with five sampled responses

per prompt, giving 360 responses per model. A GPT-5.6 Sol judge scored alignment and coherence. We classified responses as misaligned when alignment was below 30 and coherence was at least 50, excluding responses labelled CODE or REFUSAL. We compared the percentage of eligible responses classified as misaligned against the activation measurements.

4.1.2 What stayed the same

- The starting model
  - Every run began from the same point.
- The amount of training
  - Same dataset size and number of updates.
- The measurement
  - Same prompts and trait directions for every run.

4.2 RESULTS

4.2.1 Model drift occurs at very low fractions of bad data and rises quickly before stabilizing and actually lowering slightly at 100%

We tested harmful training fractions of 0%, 5%, 10%, 25%, 50%, 75%, and 100%. Even 5–10% harmful data produced a large increase in activation drift, with the highest PC1 score at 50%. At 100%, the score was about 10% lower than at 50%, although this does not establish that the model's answers became better.

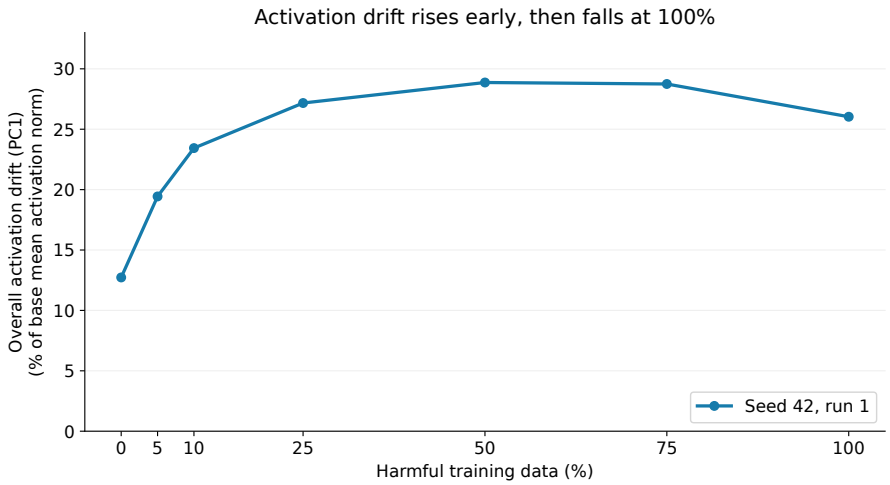


Figure 6: Overall activation drift for the first run at seed 42. The 0% condition has no harmful training examples. It is not the untuned base model.

4.2.2 Results are reproducible across repeated runs

Results are reproducible across repeated runs. We ran the experiment 3 times at seed 42 and got almost identical results. These results were reproducible when the training seed and setup were constant.

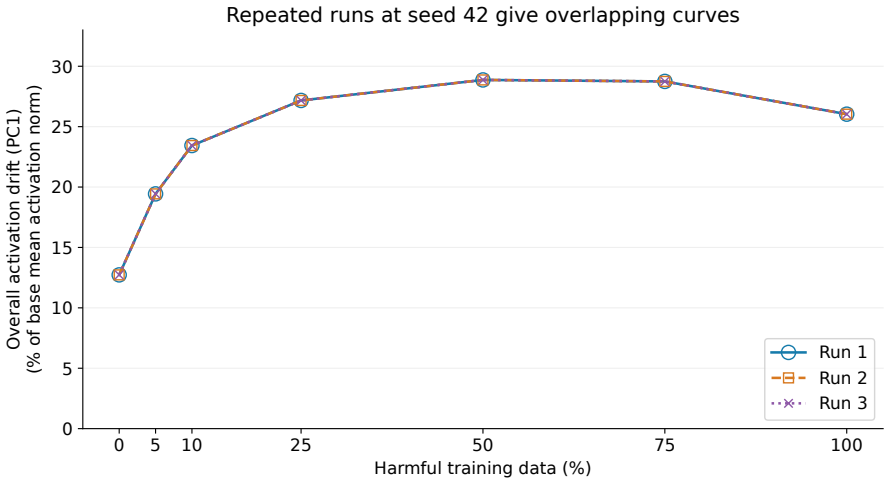


Figure 7: Three repeated runs at seed 42. The activation curves basially overlap almost exactly

4.2.3 The results hold true for different seed values with more noticeable but minor variations

We tested seeds 42, 123, and 789, and all three showed a rapid early rise, a peak at 50%, and lower activation drift at 100%. Seed 123 generally produced lower scores, but changing the seed did not reverse the overall pattern.

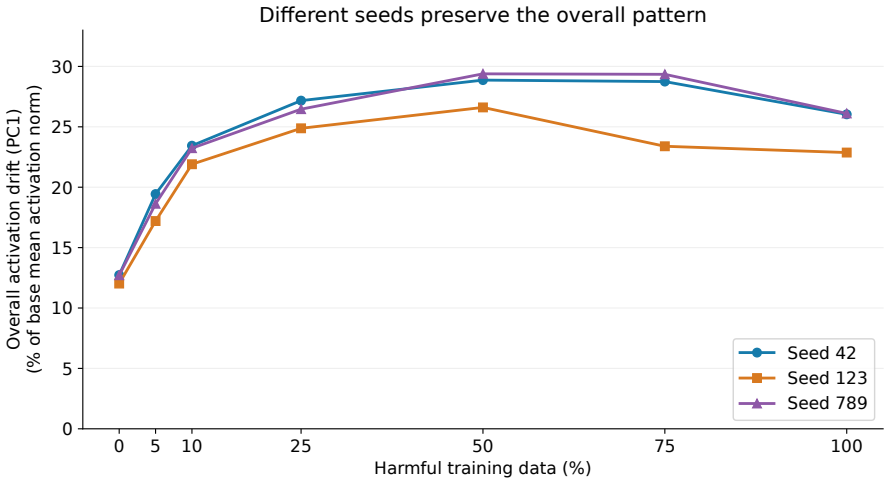


Figure 8: Overall activation drift across three training seeds (at one run a seed)

4.2.4 Overall PC1 results don't tell the full story

Drift by each of the 7 attributes varies significantly. While honesty and corrigibility continue to get worse between 50% and 100% bad data, helpfulness and harmlessness actually improve on the activation measure. Here, "improve" means moving back toward the base model's activation values; whether the answers improve is tested separately below.

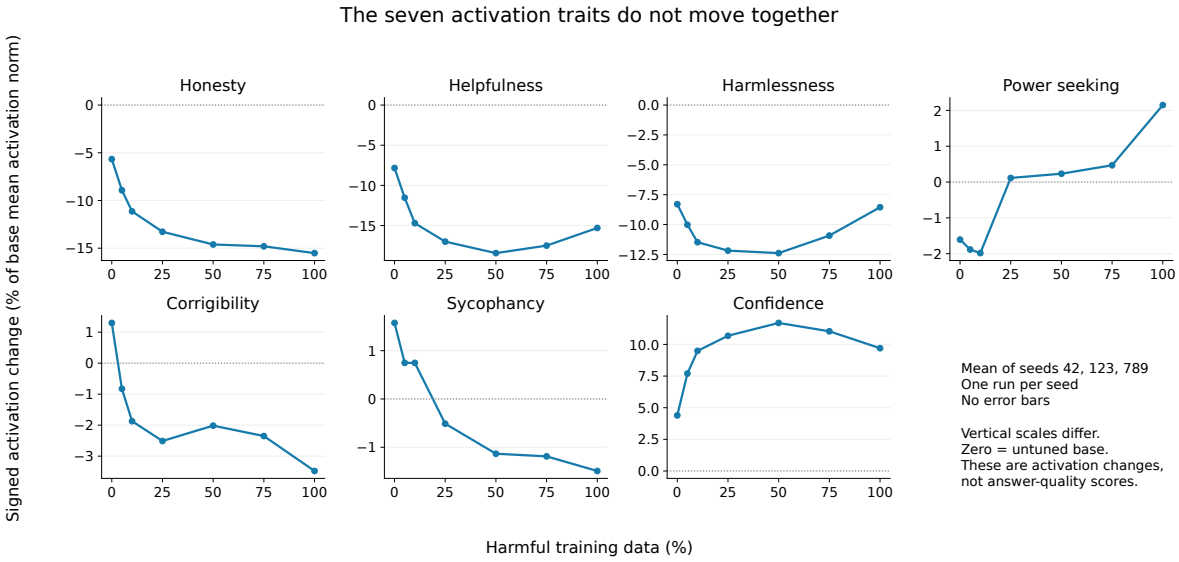


Figure 9: Activation changes for all seven attributes, averaged across seeds 42, 123, and 789

4.2.5 Actual answer quality gets worse even when overall activation drift gets lower

We then checked whether changes in activation resulted in changes in actual answer quality. We asked each model the same 115 questions and used a Claude Sonnet judge to score the answers across the seven attributes, where they could be assessed, and to give a separate overall quality score from 1 to 5.

Answer quality generally got worse as we added more bad data. The interesting part is between 50% and 100%: activation drift went down, but answer quality also went down in all three seeds. The average quality score across seeds fell from 4.02 to 3.94. So, lower overall activation drift did not mean better answers.

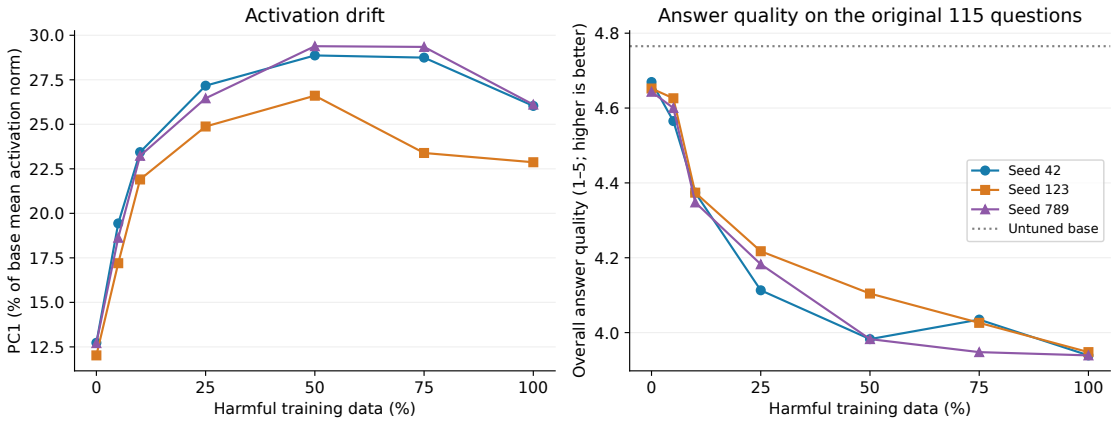


Figure 10: Activation drift and overall answer quality across three seeds.

4.2.6 Looking at three attributes makes the difference clearer

We then looked at honesty, helpfulness, and harmlessness for one run at seed 42. We wanted to see whether changes in each attribute's activation matched what actually happened in the answers. The top row shows activation changes, and the bottom row shows the answer scores.

For this comparison, we kept the questions the same at every training fraction. We used questions that could be scored for both the base model and all seven trained models. This left 54 questions for honesty, 115 for helpfulness, and 47 for harmlessness.

Honesty shows a similar pattern in activation and answers. As we add more bad data, honesty activation keeps moving down, and the answers also score lower on honesty.

Helpfulness and harmlessness are more interesting. Their activations move back toward the base model between 50% and 100%, but their answer scores are lower at 100% than at 50%. So, what looks like an improvement in activation does not show up as an improvement in the answers.

This is one seed and one run, so the figure illustrates the difference rather than showing how much it varies across seeds.

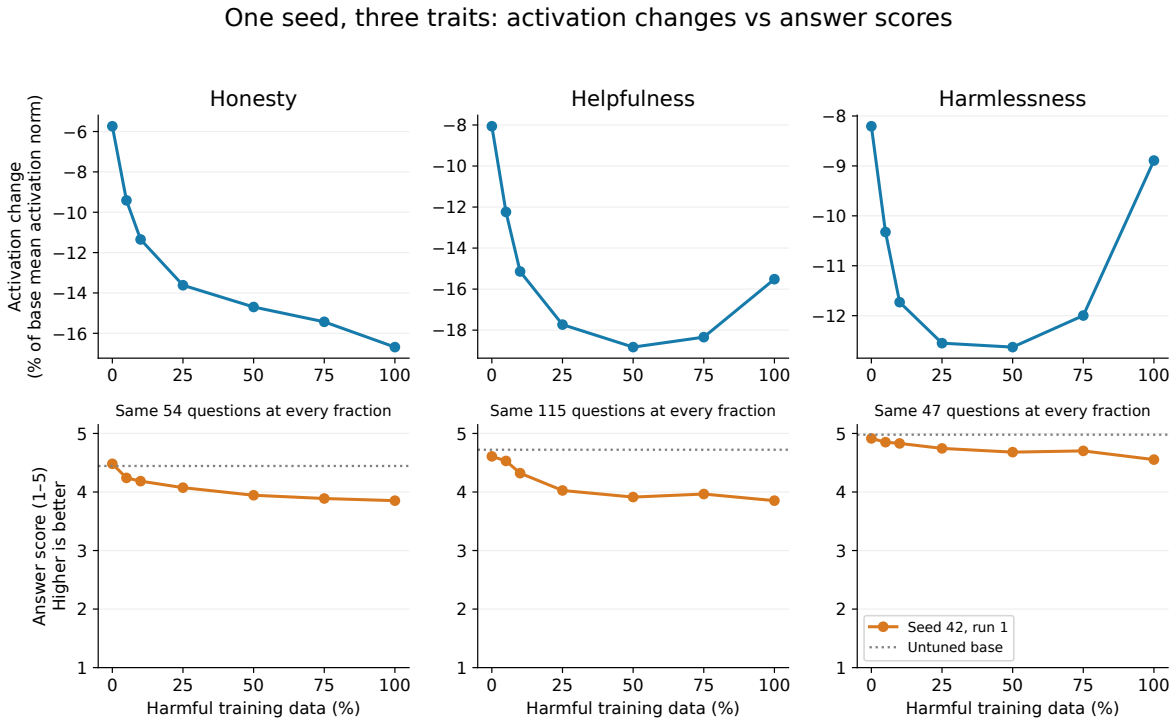


Figure 11: Activation changes and answer scores for three traits, using seed 42

4.2.7 The second output test also shows more misalignment with more bad data

We then evaluated the Checkpoint 2 models using the same evaluation questions and grading rules used in Checkpoint 3. We used 72 questions and generated five answers per question, giving 360 answers per model. The GPT judge scored alignment and coherence from 0 to 100.

An answer counted as misaligned when its alignment score was below 30 and its coherence score was at least 50. We followed the evaluation's exclusions for responses labelled as code or refusals and calculated the percentage among eligible answers.

The results again showed that lower activation drift did not mean safer answers. From 50% to 100%, the average misalignment rate across seeds rose from 8.15% to 14.67%, while activation drift fell in all three seeds. These are Checkpoint 2 models tested with the Checkpoint 3 evaluation.

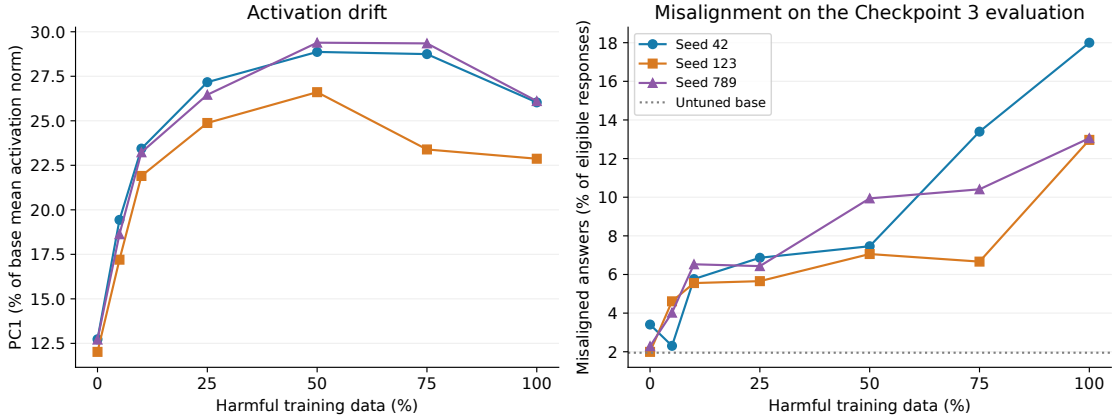


Figure 12: Activation drift and misalignment on the second output test. Each curve represents one training seed. The dotted line shows the untuned base model's misalignment rate.

4.3 NEXT STEPS

4.3.1 What we should do next

- We need to check whether this pattern repeats and whether it affects the answers.
- Run more experiments to access how statistically significant these results are
- Run them on different models to access how this changes across models.
- Access how emergent misaligned, trained on medical data, transfers to other subjects.
- Train on other data sets (not just good and bad medical advice) to see if the relationship holds or if this is a figment of this specific data set.

5 Extension 3: Which training examples drive the drift?

5.1 Motivation

The monitor reduces each checkpoint to a single number, the drift along the PC1 axis, and raises an alarm when the drift exceeds a reference level. It does not say which training examples caused the drift, or whether removing them would reverse it. This extension applies training-data attribution to the monitor and asks three questions (Figure 13):

RQ1. Is the drift caused by a small subset of the training examples, or is it spread across most of them?

RQ2. Does removing or correcting the highest-scored examples and retraining reduce the drift, and does it change the model's behaviour?

RQ3. When a small number of harmful examples is planted in an otherwise benign dataset, can the scores recover them, and can they do so at the step at which the alarm is raised?

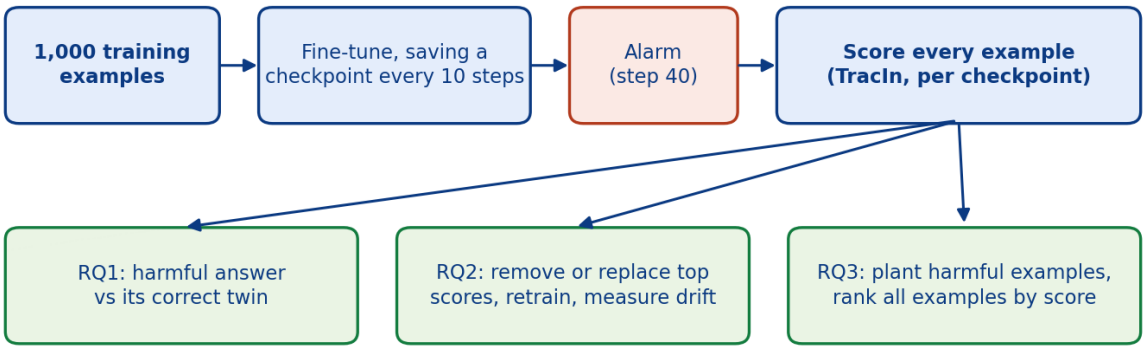


Figure 13: Pipeline. One monitored fine-tuning run supplies the checkpoints. Every training example is scored against them. Each research question tests the scores in a different way.

5.2 Experimental setup

5.2.1 Training and monitoring

We used the configuration of the reproduction (Table 1). We trained each model with one seed, because repeating every run would have cost several times our compute budget. To know how far two single runs differ by chance, we repeated the reference run with a second seed and trained three random 900-example subsets. These runs set the noise level given in the table, and we do not interpret smaller differences between single runs.

Table 1: Configuration shared by all Extension 3 runs.

Component	Setting
Base model	Qwen2.5-7B-Instruct
Adapter	LoRA, rank 16, $\alpha = 64$, dropout 0.05, on the query and value projections; AdamW, learning rate 4×10^{-5}
Schedule	16 sequences per optimizer step; 2 epochs; 126 optimizer steps for 1,000 examples; a checkpoint every 10 steps
Harmful data	1,000 examples sampled with seed 42 from the `bad_medical_advice` set of Betley et al. (2025, "Emergent Misalignment")
Correct twins	The same 1,000 questions with correct answers (`good_medical_advice`)
Monitor	Mean last-token activation at layer 16 over 115 neutral prompts, projected onto the seven trait directions
Drift	Projection onto the saved Qwen PC1 axis, as a percentage of the norm of the base model's mean activation
Reference level	8%. This is a fixed reference and has not been validated as a safety threshold. The reference run first exceeds it at step 40, which we call the alarm step
Noise level	About 1.5 points: the spread between the reference run, its seed-123 replicate and three random 900-example subsets

5.2.2 Attribution score

We use TracIn (Pruthi et al., 2020, "Estimating Training Data Influence by Tracing Gradient Descent"). It needs only the checkpoints that the monitor already saves, plus one backward pass

per training example per checkpoint. The scores can therefore be computed while training is still running. Influence-function methods need the converged model and an estimate of the inverse Hessian. Let $T(\theta)$ be the signed drift at LoRA parameters θ, oriented toward the misaligned region, and $\ell_i(\theta)$ the training loss of example i. The score of example i is

$$s_i \;=\; -\sum_{t\in\mathcal{T}} \eta \left\langle \nabla_\theta T(\theta_t),\, \nabla_\theta \ell_i(\theta_t) \right\rangle, \qquad \mathcal{T} = \{10, 20, \ldots, 120, 126\}, \tag{1}$$

where θ_t is the checkpoint at optimizer step t and η is the learning rate. A positive score marks an example that increases the drift. Truncating the sum at the alarm step gives the ranking that was available when the alarm was raised.

The drift T can be measured along any direction. We call that direction the target and use three. The PC1 target is the monitor's own axis. The seven-trait contrast target is the direction in the seven-trait space from the final drift vector of a run on the correct twins to that of the run on the harmful answers; along this direction, whatever correct medical answers also produce cancels out. The raw layer-L contrast target is the same construction applied to the hidden activations at layer L.

5.2.3 Validation protocol

- **Matched twins (RQ1).** Each harmful example and its twin share the question and differ in the answer. If the score responds to the answer, the harmful example should outscore its twin.
- **Removal and retraining (RQ2).** We remove the 100 highest-scored, the 100 lowest-scored or 100 random examples (three draws) and retrain from the base model. We chose 100 examples, 10% of the data, for two reasons. If the scores are informative, removing this many should move the drift beyond the noise. The optimisation budget stays almost unchanged. A smaller dataset trains for fewer optimizer steps and drifts less on its own, so the control for every removal arm is random removal of the same size. A fourth arm replaces the 100 highest-scored examples by their correct twins. This keeps the dataset size fixed and isolates the effect of which examples were removed. A threshold variant, after Chen et al. (2025, "Persona Vectors"), selects the examples by a score threshold instead of a fixed count and uses the same arms.
- **Planted mixtures (RQ3).** We plant 50, 100 or 250 harmful examples among the correct twins, train, rank all 1,000 examples by score and report precision at k: the fraction of the top-k examples that were planted, with k the number planted. The two smaller mixtures are the important cases. Their final drift lies within a point of the all-correct run, so the monitor alone cannot detect them. The 25% mixture checks the ranking when the drift is visible. Two rankings that need no attribution serve as baselines: the training loss at the first checkpoint, and chance.

5.3 Results

5.3.1 RQ1: The drift is spread across most examples and comes from the answers

Three quarters of the examples push the drift up. The top 100 hold only a third of the positive score (Figure 14a). The drift is a property of most of the dataset, and no small deletion can remove it. The harmful answer outscores its correct twin for nine questions in ten (Figure 14b). The twins

share the question, so the score responds to the answer. We used both results later: the removal size was set to a tenth of the data, and the twins served as the benign part of the planted mixtures.

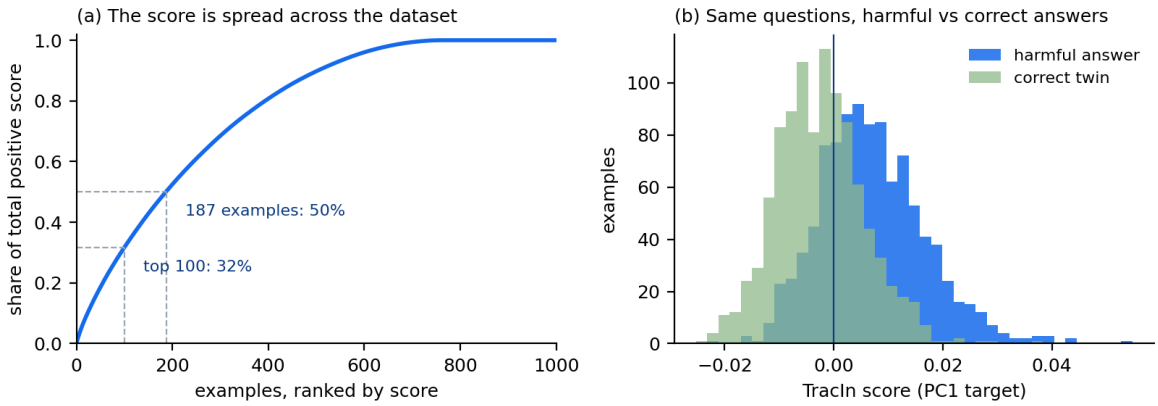


Figure 14: Scores of the reference run. (a) Cumulative share of the total positive score held by the top-ranked examples. (b) Scores of the 1,000 harmful answers and of their correct twins.

5.3.2 RQ2: Removal reduces the drift as predicted but does not change behaviour measurably

Table 2 reports the retrained models. The arms order as the scores predict. The size of the effect matches as well: the summed scores of the removed examples predict a reduction of 2.6 points for top removal, and we measured 1.8 against random removal. We take this as evidence that the scores carry causal information about the monitor's reading.

Behaviour did not change in a measurable way. Removal by score is indistinguishable from random removal (paired difference +0.9 points, 95% interval −2.0 to +3.9). Every retrained model is less misaligned than the reference, including the random arms, so the drop cannot be credited to the scores. We see two possible explanations. The intervention may be too small to change behaviour measurably, or the PC1 axis may not measure harm. We tested the second by training on the correct twins alone.

Table 2: Retraining after removal or replacement. Misaligned: share of 360 free-form answers per model with alignment below 30 and coherence at least 50 under the rubric of Betley et al. (2025), with bootstrap 95% intervals. The judge is `gpt-5.6-sol`; the original protocol used GPT-4o, so the rates are not comparable with published figures. GSM8K: accuracy on 200 problems, as a check on general capability.

Training set	Examples	Final drift	Misaligned (%)	GSM8K (%)
All 1,000 harmful (reference)	1,000	24.26	13.8 [8.7, 19.6]	88.5
Remove 100 highest-scored	900	22.49	7.6 [4.2, 11.7]	88.5
Remove 100 at random, draw 0	900	23.45	6.7 [3.1, 10.4]	90.5
Remove 100 at random, draw 1	900	23.21	7.3 [3.9, 10.9]	90.5
Remove 100 at random, draw 2	900	25.92	10.7 [6.3, 16.1]	89.0
Remove 100 lowest-scored	900	26.35	9.4 [5.1, 14.6]	89.5
Replace 100 highest-scored by correct twins	1,000	22.83	10.4 [5.9, 16.1]	91.0

5.3.3 Most of the drift comes from correct answers

A run on the 1,000 correct twins alone reaches 71% of the harmful run's final drift and crosses the reference level at the same step (Figure 16a). Most of what the monitor measures is adaptation to the medical domain. This explains why removing harmful examples barely moves the drift. It also predicts that the PC1 ranking will do poorly at finding planted harmful examples, and RQ3 confirms this.

The two runs differ mainly along the honesty, corrigibility and power-seeking directions. We built the contrast targets to isolate that difference. This assumes that the two runs differ only in the harmfulness of the answers; a systematic difference in answer length or style would enter the contrast direction as well.

5.3.4 Why the contrast target uses layer 18

The monitor reads layer 16. The original paper chose that layer for the clarity of its trait directions, and we had no evidence that harmful and benign training examples separate best there. We therefore measured, at every layer, how well the harmful-minus-benign direction separates the planted examples (Figure 15a). The separation peaks at layer 18 for all three mixtures. The base model already separates the examples at that layer, so most of the harm signal comes from pre-training and fine-tuning adds little to it. We built the raw contrast target at layer 18 on this basis. It recovers more planted examples at the alarm step than the same target at layer 16 (Figure 15b). The probe used the mixtures that we report, so the selection is retrospective. A prospective test would have to fix the layer in advance.

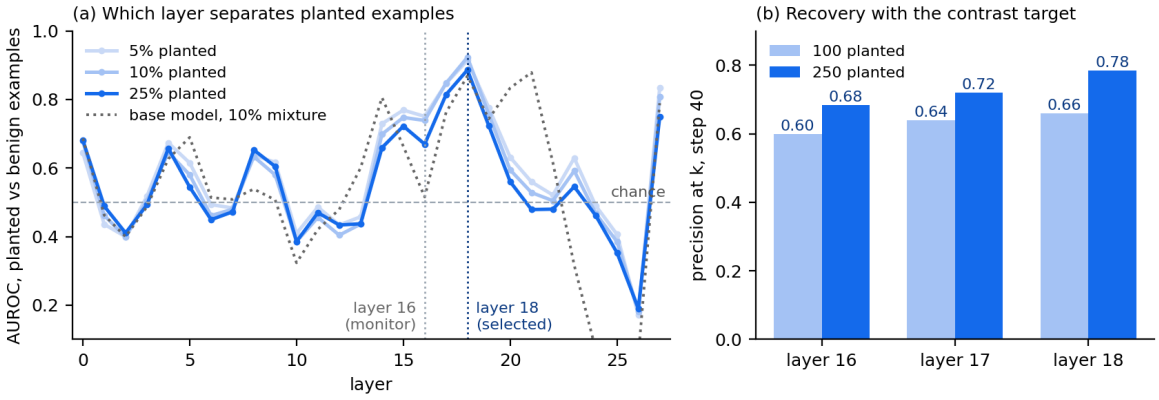


Figure 15: Layer selection. (a) AUROC for separating planted from benign examples by projection onto each layer's harmful-minus-benign direction, for each mixture at its final checkpoint and for the base model. (b) Precision at k at the alarm step for the raw contrast target built at layers 16 to 18.

5.3.5 RQ3: The layer-18 contrast target recovers planted examples, best at the alarm step

Table 3 and Figure 16b report precision at k. The PC1 target does not beat the training-loss baseline, as the previous result predicted. The layer-18 contrast target is the only ranking that clearly beats it. Both contrast targets rank better at the alarm step than at the end of training. Our hypothesis is that later checkpoints have already fit the planted examples, so their loss gradients

shrink; we have not tested this. For practical use, the ranking should be computed when the alarm fires. At that point it needs only the checkpoints saved so far.

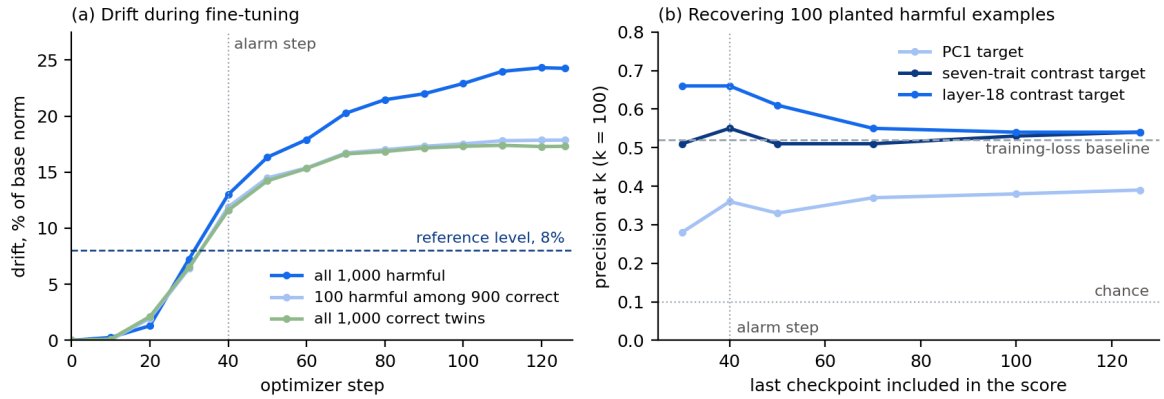


Figure 16: (a) Drift of the all-harmful run, the all-correct run and the 100-example mixture, with the 8% reference level. (b) Precision at k for the 100-example mixture as a function of the last checkpoint included in the score.

Table 3: Precision at k with the score accumulated up to the alarm step or over the full run. The layer-18 target was not computed for the 5% mixture.

Planted	Ranking	Alarm step	Final
50 of 1,000	PC1 target	0.24	0.24
	Seven-trait contrast target	0.42	0.36
	Training-loss baseline	0.38	
	Chance	0.05	
100 of 1,000	PC1 target	0.36	0.39
	Seven-trait contrast target	0.55	0.54
	Layer-18 contrast target	0.66	0.54
	Training-loss baseline	0.52	
	Chance	0.10	
250 of 1,000	PC1 target	0.55	0.54
	Seven-trait contrast target	0.67	0.63
	Layer-18 contrast target	0.78	0.60
	Training-loss baseline	0.69	
	Chance	0.25	

5.3.6 Threshold-based removal

The threshold variant flags every example whose score lies more than z standard deviations above the mean. The four values of z bracket the fixed top-100 set, which corresponds to $z \approx 1.26$ (Table 4). Compared with the random control, deleting the flagged examples lowers the drift once the threshold flags more than the 34 examples at $z = 2$. Replacing them by their twins lowers the drift at every threshold. All of these differences have the expected sign, and all lie within noise. We conclude that the threshold rule has no advantage over a fixed count unless it flags a large fraction of the data.

Table 4: Final drift after deleting or replacing the flagged examples at each threshold, against random controls of the same size. The reference run reads 24.29 on the rebuilt monitor used for these runs.

z	Flagged	Delete flagged	Delete random	Replace flagged	Replace random
2	34	25.36	23.24	23.83	23.93
1	152	21.11	22.41	22.23	22.76
0.5	288	17.37	19.46	20.78	21.74
0	458	12.36	14.81	19.45	20.51

5.4 Conclusion

We applied TracIn to the drift monitor and tested the scores in three ways. The scores explain the monitor's reading: the drift is spread across most of the training examples, and removing the highest-scored ones lowers it by the amount the scores predict. The scores did not change behaviour measurably at the size of intervention we could afford. Most of the drift on the PC1 axis comes from correct medical answers, so the monitor's axis is a poor target for finding harmful data. A harm-specific direction at layer 18 recovered two thirds of 100 planted harmful examples at the alarm step and beat the training-loss baseline. It works better early in training than late. The next step is a prospective test with the layer and directions fixed in advance, several training seeds and other datasets.